Chapter 7. 업무 자동화 Agent (강사 데모)

> ⏱️ **배정 시간 30분** — 설계 관점 10분 · 시연 15분 · 정리 5분

> 📺 **이 챕터는 강사 시연으로 진행합니다.** 이메일 자동화는 개인 계정 설정(앱 비밀번호, 메일 도착 대기)에 따라 진행 속도 차이가 크고, 실습실 네트워크에서 IMAP(993 포트)이 막혀 있을 수 있기 때문입니다. 화면으로 전체 흐름을 함께 확인하고, **수업 후 각자 그대로 따라 만들 수 있도록 모든 설정값을 아래에 기재**해두었습니다.

📋 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다.

- [ ] 백그라운드 자동화와 대화형 Agent의 설계 차이를 설명할 수 있다
- [ ] 언제 AI에게 판단을 맡기고 언제 IF 노드로 통제해야 하는지 구분할 수 있다
- [ ] Email Trigger(IMAP)로 새 이메일을 감지하는 구조를 이해할 수 있다
- [ ] Chapter 3의 태그 출력 방식이 실제 후처리에 어떻게 쓰이는지 확인할 수 있다
- [ ] (수업 후) 교안을 보고 이 워크플로우를 직접 재현할 수 있다

📖 이 장의 용어

용어	영문 · 약어 풀이	설명
백그라운드 자동화	Background automation	사람이 화면을 보지 않는 동안 혼자 동작하는 자동화. 대화형과 달리 예측 가능성이 중요
IMAP	Internet Message Access Protocol	메일함의 메일을 읽어 오는 통신 규약. 보낼 때는 SMTP, 받을 때는 IMAP. Gmail은 `imap.gmail.com`, 포트 993
INBOX	받은 편지함	IMAP에서 기본 메일함의 이름
폴링	Polling	새 메일이 있는지 일정 간격으로 반복해 확인하는 방식. Email Trigger가 이렇게 동작
Post-Processing Action	후처리 동작	메일을 읽은 뒤 어떻게 할지. Nothing(그대로) / Mark as Read(읽음 처리). 운영에서는 반드시 Mark as Read

용어	영문 · 약어 풀이	설명
분류	Classification	입력을 정해진 카테고리 중 하나로 나누는 작업. 결과가 일정해야 하므로 Temperature를 낮게
파싱	Parsing	텍스트에서 필요한 부분을 규칙에 따라 뽑아내는 것. Code 노드가 태그 안의 값을 파싱
회신 초안	Reply draft	사람이 다듬어 보내는 것을 전제로 AI가 미리 써 둔 답장. 자동 발송하지 않음
정규식	Regular Expression	문자 패턴을 찾는 표기법. Chapter 3과 같은 `pick` 함수 사용

7.1 설계 관점: 누가 판단할 것인가

7.1.1 시나리오

고객 문의 메일함에 이메일이 쌓이면 매번 열어서 내용을 파악하고 담당자에게 전달하는 일은 번거롭습니다. 이 프로젝트는 새 이메일이 도착할 때마다 자동으로 아래를 수행합니다.

1. 이메일 내용을 2줄로 요약
2. "문의 / 불만 / 제안 / 기타" 중 하나로 분류
3. 담당자가 다듬어 보낼 수 있는 **회신 초안** 작성
4. 결과를 Data Table에 기록
5. "불만"으로 분류된 경우에만 담당자에게 즉시 알림

7.1.2 Chapter 6과 무엇이 다른가

여기가 이 챕터에서 가장 중요한 부분입니다.

Chapter 6의 헬프데스크 Agent는 **대화형**이었습니다. 사람이 화면 앞에 있고, 답변이 이상하면 다시 물어볼 수 있습니다. 그래서 에이전트가 상황에 따라 스스로 Tool 호출 여부를 판단하도록 맡겼습니다.

이번 프로젝트는 **사람이 지켜보지 않는 백그라운드 자동화**입니다. 새벽 3시에 메일이 와도 혼자 돌아가야 하고, 잘못 처리되어도 아무도 즉시 알아채지 못합니다. 이런 경우에는 설계를 반대로 가져갑니다.

구분	Chapter 6 (대화형)	Chapter 7 (백그라운드)
AI의 역할	판단 + 도구 선택	**내용 이해만** (요약·분류·초안)
실행 경로 결정	AI가 스스로	**IF 노드가 확실하게**
우선 가치	유연함	**예측 가능성**

구분	Chapter 6 (대화형)	Chapter 7 (백그라운드)
사람의 개입	대화 중 즉시	기록을 보고 사후에

AI에게는 "이메일을 읽고 요약·분류·초안 작성"이라는 이해 작업만 시키고, **그 결과를 가지고 무엇을 할지는 IF 노드로 명확하게 통제**합니다. 결과를 예측하기 어려운 자율 판단보다, 정해진 경로로 처리되는 쪽이 운영 안정성에서 유리하기 때문입니다.

> 💡 Chapter 1에서 다룬 "단순 자동화 / LLM Chain / Agent" 구분을 떠올려보세요. 이 프로젝트는 **LLM Chain**에 가깝습니다. Agent가 항상 정답은 아니며, **무엇을 만드느냐에 따라 적절한 구조가 다릅니다.** 이 판단이 이 챕터에서 가져갈 가장 중요한 것입니다.

> ⚠️ **회신 초안을 자동 발송하지 않는 이유**도 같은 맥락입니다. AI가 쓴 문장이 고객에게 바로 나가면 잘못된 안내가 되돌릴 수 없는 결과를 만듭니다. 초안은 사람이 확인하고 보내는 것을 전제로 기록만 합니다.

7.1.3 노드 구성

```
Email Trigger(IMAP)
        │
        ▼
Basic LLM Chain (요약 + 분류 + 회신 초안)
        │
        ▼
Code (응답에서 세 값 추출)
        │
        ▼
Data Table (항상 기록)
        │
        ▼
IF (카테고리 == "불만"?)
        │
        ├── true → 담당자 알림 발송
        └── false → 종료
```

✅ 체크포인트 질문

- AI 판단과 IF 노드 판단의 역할을 나눈 이유는? → **사람이 지켜보지 않는 자동화에서는 실행 경로가 예측 가능해야 하기 때문**
- Data Table 기록을 IF 분기 **앞에** 둔 이유는? → **카테고리와 상관없이 모든 이메일을 빠짐없이 기록해야 하기 때문**
- 회신 초안을 자동 발송하지 않는 이유는? → **AI가 쓴 문장이 검토 없이 고객에게 나가면 잘못된 안내를 되돌릴 수 없기 때문**

7.2 시연 — 전체 흐름 따라가기

> 👀 수업 중에는 눈으로 흐름을 따라가는 데 집중하시고, 재현은 수업 후에 하시면 됩니다. 아래에 모든 설정값이 그대로 적혀 있습니다.

7.2.1 Email Trigger 연결

사전 준비

1. Gmail 앱 비밀번호 — Chapter 2에서 발급받은 것을 그대로 사용합니다
2. Gmail IMAP(Internet Message Access Protocol, 메일함을 읽어 오는 규약) — 2025년 1월부터 Gmail은 IMAP이 항상 켜져 있고 사용/사용 안함 설정이 없어졌습니다. 개인 Gmail 계정은 별도 설정 없이 진행합니다. 회사 Google Workspace 계정은 관리자가 IMAP을 막아 둘 수 있습니다.

노드 설정

1. 새 워크플로우 생성, 이름을 `시연7_문의메일분류`로 설정
2. `Email Trigger (IMAP)` 노드 추가하고 아래와 같이 Credential 생성
3. `Mailbox`는 기본값 `INBOX` 유지
4. `Post-Processing Action`을 `Nothing`으로 설정

항목	값
Host	`imap.gmail.com`
Port	`993`
User	본인 Gmail 주소
Password	Chapter 2의 앱 비밀번호 16자리
SSL/TLS	사용함

> ⚠️ `Post-Processing Action`을 `Nothing`으로 두는 이유: 실습 중에는 같은 메일로 반복 테스트해야 하므로 읽음 처리를 하지 않습니다. **실제 운영에서는 반드시 `Mark as Read`로 바꿔야 합니다.** 그렇지 않으면 같은 메일이 계속 감지되어 무한히 처리됩니다.

동작 확인

Email Trigger는 Schedule Trigger와 마찬가지로 **워크플로우가 Publish된 상태에서만** 자동 감지합니다.

1. 우측 상단 **Publish** 클릭

2. 연결한 Gmail 주소로 테스트 메일 발송 (제목 `배송 문의`, 내용 `주문한 상품이 아직도 배송 준비 중인데 언제 오나요?`)
3. 잠시 뒤 **Executions 탭**에서 새 실행이 기록되는지 확인
4. Email Trigger 노드의 OUTPUT에서 `subject`(제목), `textPlain`(본문), `from`(발신자) 필드가 들어오는지 확인

> 💡 Email Trigger의 기본 출력 형식(Simple)에서 메일 본문은 `text`가 아니라 `textPlain`(일반 텍스트)과 `textHtml`(HTML) 필드에 담깁니다. 편집 화면에서 뒤쪽 노드를 ▶로 실행하려면 먼저 Email Trigger 노드를 열어 **Fetch Test Event**로 테스트 메일을 불러와야 합니다.

7.2.2 요약·분류·회신 초안

1. Email Trigger 뒤에 `Basic LLM Chain` 노드 추가
2. `Google Gemini Chat Model` 연결 (Flash 계열, Temperature `0.2`)
3. `Prompt`를 `Define below`로 변경 (Chapter 3에서 반복 강조한 그 설정입니다)
4. `Prompt (User Message)`에 아래 내용 입력
5. ▶ 버튼을 눌러 OUTPUT에 `<summary>`, `<category>`, `<reply_draft>` 태그가 포함된 응답이 나오는지 확인

```
당신은 고객 문의 메일을 1차 처리하는 담당자입니다.
다음 <email> 태그 안의 이메일을 처리하세요.

<email>
제목: {{ $json.subject }}
내용: {{ $json.textPlain }}
</email>

1. 이메일 내용을 2줄 이내로 요약하세요.
2. 아래 4개 카테고리 중 하나로 정확히 분류하세요: 문의, 불만, 제안, 기타
   반드시 이 4개 단어 중 하나만 사용하고, 다른 표현을 쓰지 마세요.
3. 담당자가 다듬어 보낼 회신 초안을 정중한 문체로 3문장 이내로 작성하세요.
   확인이 필요한 사실(배송 상태, 주문 정보 등)은 단정하지 말고 "확인 후 안내드리겠습니다"로 작성하세요.

결과는 아래 형식을 반드시 지켜서 출력하세요.
<summary>(요약 내용)</summary>
<category>(문의 또는 불만 또는 제안 또는 기타 중 하나만)</category>
<reply_draft>(회신 초안)</reply_draft>
```

응답 파싱 (Code 노드)

1. 뒤에 `Code` 노드를 추가하고 아래 코드 입력

2. ▶ 버튼을 눌러 `subject`, `from`, `summary`, `category`, `reply_draft` 다섯 필드가 깔끔하게 정리되어 나오는지 확인

```
const raw = $json.text;

const pick = (tag) => {
  const match = raw.match(new RegExp('<' + tag + '>([\\s
\\S]*?)</' + tag + '>'));
  return match ? match[1].trim() : '';
};

return [{
  json: {
    subject: $('Email Trigger (IMAP)').item.json.subject,
    from: $('Email Trigger (IMAP)').item.json.from,
    summary: pick('summary') || '요약 실패',
    category: pick('category') || '기타',
    reply_draft: pick('reply_draft')
  }
}];
```

> ⚠️ `$('Email Trigger (IMAP)')`의 노드 이름은 **캔버스에 있는 실제 노드 이름과 정확히 일치**해야 합니다. 노드 이름을 바꿨다면 코드도 함께 고쳐야 합니다. `pick` 함수는 Chapter 3 실습 3-2에서 쓴 것과 같습니다.

7.2.3 Data Table 기록과 조건부 알림

Data Table 준비

사이드바 **Data tables**에서 `이메일처리기록` 테이블을 만들고 컬럼을 추가합니다 (Chapter 6의 6.2.2와 같은 방법).

컬럼명	타입
`received_at`	String
`sender`	String
`summary`	String
`category`	String
`reply_draft`	String

> ⚠️ Data Table의 컬럼명은 영문·숫자·밑줄(_)만 허용됩니다(Chapter 6 참고). 각각 날짜, 발신자, 요약, 카테고리, 회신초안을 뜻합니다.

Data Table 노드

1. Code 노드 뒤에 `Data Table` 노드 추가, Operation은 `Insert row`, 테이블은 `이메일처리기록` 선택
2. 컬럼 매핑

컬럼	값
received_at	`{{ $now.setZone('Asia/Seoul').toFormat('yyyy-MM-dd HH:mm') }}`
sender	`{{ $json.from }}`
summary	`{{ $json.summary }}`
category	`{{ $json.category }}`
reply_draft	`{{ $json.reply_draft }}`

IF 노드로 긴급 알림 분기

1. Data Table 뒤에 `IF` 노드 추가
2. 조건 설정 (타입 **String**) — Value 1은 `{{ $('Code').item.json.category }}`, Operator는 `is equal to`, Value 2는 `불만`
3. `true` 출력에 알림 노드(`Send Email` 또는 Slack용 `HTTP Request`) 연결
4. `false` 출력은 연결 없이 종료

> ⚠️ **여기서 `$json.category`가 아니라 `$('Code')...`를 쓰는 이유**: IF 노드의 바로 앞은 Data Table 노드입니다. 저장 노드를 거치면 출력 필드가 **그 노드가 돌려주는 형태(id, 생성 시각, 컬럼명 등)** 로 바뀌어, 원래 필드가 사라지거나 이름이 달라질 수 있습니다. 그래서 값을 만든 Code 노드의 출력을 직접 참조합니다. 노드를 거칠 때마다 필드가 어떻게 바뀌는지 OUTPUT 패널로 확인하는 습관이 필요합니다.

```
[긴급] 불만 이메일 접수
발신자: {{ $('Code').item.json.from }}
요약: {{ $('Code').item.json.summary }}

회신 초안:
{{ $('Code').item.json.reply_draft }}
```

최종 테스트

Gmail로 불만 성격의 메일을 보냅니다. Data Table에 행이 기록되고(회신초안 포함), 담당자에게 긴급 알림이 도착하는지 확인합니다.

```
제목: 배송 지연 불만
내용: 주문한 지 2주가 지났는데 아직도 안 왔어요. 너무 화가 납니다.
```

7.3 자주 발생하는 오류 (수업 후 재현 시 참고)

증상	원인	해결
새 이메일을 감지하지 못함	워크플로우가 Publish되지 않음	우측 상단 Publish 상태 확인
IMAP 인증 오류	앱 비밀번호 오류, 또는 Workspace 계정에서 관리자가 IMAP을 차단	앱 비밀번호 재발급, 개인 Gmail 계정 사용
연결 자체가 안 됨 (timeout)	네트워크에서 993 포트 차단	개인 네트워크에서 재현, 또는 네트워크 담당자에게 문의
같은 메일이 계속 반복 처리됨	`Post-Processing Action`이 `Nothing`	운영 시에는 `Mark as Read`로 변경
요약이 메일 내용과 무관하거나 "내용이 없다"고 나옴	프롬프트에서 본문을 `$json.text`로 참조	본문 필드는 `$json.textPlain`
태그가 응답에 없음	`Prompt`가 `Define below`로 안 바뀜	Basic LLM Chain의 Prompt 옵션 확인
Code 노드에서 undefined 오류	참조한 노드 이름이 실제와 다름	캔버스의 노드 이름을 확인하고 코드 수정
Data Table에 빈 값으로 기록됨	정규식이 모델 응답 형식과 안 맞음	Basic LLM Chain의 실제 OUTPUT에서 태그 형식 확인
카테고리가 4개 외의 단어로 나옴	프롬프트의 제한 지시가 약함	"반드시 이 4개 단어 중 하나만" 문구를 더 강조
IF가 항상 false	카테고리 값에 공백·줄바꿈 포함	Code 노드의 `.trim()` 확인, OUTPUT에서 실제 값 확인

📌 Chapter 7 요약

개념	핵심 내용
설계 철학	대화형은 AI의 자율 판단, 백그라운드는 **AI의 이해 + IF의 확실한 분기**
Email Trigger(IMAP)	앱 비밀번호로 인증. **Publish 상태에서만** 자동 감지

개념	핵심 내용
Post-Processing Action	실습은 Nothing, 운영은 Mark as Read (안 바꾸면 무한 반복)
태그 기반 출력	Chapter 3에서 배운 방식으로 요약·분류·초안을 한 번에 추출
Code 노드 파싱	정규식으로 태그 안의 값을 추출. 참조 노드 이름 정확히 일치 필요
노드 통과 시 필드 변화	저장 노드를 거치면 출력 필드가 바뀜 → 원본 노드를 직접 참조
기록은 분기 앞에	조건과 무관하게 모든 건을 남겨야 하므로
회신 초안	자동 발송하지 않고 기록만. 사람이 확인 후 발송

다음 Chapter 예고: 마지막 Chapter 8에서는 만든 워크플로우를 실제 운영 상태로 올리고, 문제가 생겼을 때 추적하고 복구하는 방법을 다룬 뒤, 실습실 PC를 정리하고 각자의 업무 자동화 아이디어를 설계합니다.